Modultestbericht

Faktura — Desktop-Fakturierungsanwendung

Lucas Strubel

Version 1.3 · 18.10.2026

Inhaltsverzeichnis

Dokumentenhistorie 2
1. Einleitung 2
2. Testdurchführung 3
3. Testergebnisse 4
4. Ergebnisübersicht und Statistik 9
5. Nachtest Version 3.0 (24.09.2026) 10
6. Bewertung und Abweichungen 12
7. Abkürzungen 13
8. Review-Remediation 3.1.0-rc.1 (10.10.2026) 13
9. Startzeit Q-04 vor der Freigabe 3.1.0 (18.10.2026) 17

Autor	Rolle	Stand
Lucas Strubel	Verfasser für das SE1-Projektteam (Gesamtsystem)	24.06.2026 (Abnahme v1.0), 24.09.2026 (Nachtest v3.0)

Freigabevermerk: Dieses Dokument dokumentiert die Durchführung und die Ergebnisse des Modultests des Gesamtsystems *Desktop-Fakturierungsanwendung* (Komponenten A–D, gemeinsame Infrastruktur sowie Performance-Nachweise) gemäß Modultestplan v2.0. Es ist nach Prüfung und Freigabe durch den Modulverantwortlichen verbindlicher Nachweis des durchgeführten Modultests.

Aktueller Freigabestand (18.10.2026): 3.1.0-rc.1 ist als Vorabversion veröffentlicht. Die Startzeit Q-04 ist inzwischen mit der gebündelten Laufzeit nachgewiesen (Kapitel 9); für 3.1.0 stehen noch die Installer-Prüfung von der Vorabversion auf 3.1.0 und der Versionswechsel aus. Die historische Abnahme bleibt unten erhalten. Kapitel 8 trennt Regressionen von den damals offenen Leistungs-, Plattform-, Installations- und Sicherheitsnachweisen.

Dokumentenhistorie

Version	Datum	Autor	Grund der Änderung
1.0	24.06.2026	Lucas Strubel	Initiale Erstellung; Dokumentation der Modultest-Durchführung auf Basis des Modultestplans v2.0
1.1	24.09.2026	Lucas Strubel	Kapitel 5: Nachtest Version 3.0 (201 Ausführungen, 0 Fehlschläge, Abdeckung und SpotBugs); Kapitel 1–4 unverändert
1.2	10.10.2026	Lucas Strubel	Review-Remediation 3.1.0-rc.1; historische Nachweise und offene Freigabeschranken getrennt
1.3	18.10.2026	Lucas Strubel	Kapitel 9: Startzeit Q-04 mit gebündelter Laufzeit nachgewiesen; Messmethode, Maßnahmen und verworfene Ansätze

1. Einleitung

1.1 Zweck des Dokuments

Dieser Modultestbericht dokumentiert die **Durchführung** und die **Ergebnisse** der im Modultestplan v2.0 spezifizierten Modul-/Komponententests des **Gesamtsystems** *Desktop-Fakturierungsanwendung*. Berichtet werden die Ergebnisse aller vier Komponenten (A – Prozess/Dokumentenzyklus, B – Produktverwaltung, C – Kundenverwaltung, D – Programmoberfläche), der gemeinsam genutzten Infrastruktur (Paket `gemeinsam`) sowie der übergreifenden Performance-/Lastnachweise. Für jeden im Plan festgelegten Testfall werden die ausführende JUnit-5-Testmethode und das tatsächliche Ergebnis (`Bestanden`/`Fehlgeschlagen`) ausgewiesen. Quelle der Wahrheit sind die Testklassen unter `src/test/java/de/lucasstrubel/faktura/…` sowie die beim Testlauf erzeugten Surefire-Berichte (`target/surefire-reports/`).

> **Stand:** Dieses Dokument ist der Nachweis des Modultestlaufs zur Abnahme der **Version 1.0** (Stand 24.06.2026) und wird als historischer Beleg unverändert geführt. Alle Ergebnisse,

> Laufzeiten und Aussagen zur Oberfläche in den Kapiteln 1–4 und 6 beziehen sich auf diesen Stand — insbesondere die Angabe „ohne Swing-Rendering“ in Kapitel 6 — und geben nicht den aktuellen Stand der Anwendung wieder. Den Nachtest der Version 3.0 dokumentiert Kapitel 5.

1.2 Testgegenstand und Referenzdokumente

Prüfgegenstand ist die Anwendung `fakturierung` in Version 1.0.0. Grundlage der Testfälle (Vorbedingungen, Eingaben, erwartete Ergebnisse) ist der **Modultestplan v2.0**; die fachlichen Anforderungen stammen aus den Teilen A–D des Pflichtenhefts sowie dem Lastenheft v1.3. Die Abkürzungen für die abgedeckten Anforderungen (Spalte *Abgedeckte Anf.*) sind — wie im Modultestplan — **komponentenlokal** zu lesen: `F-01` in Abschnitt 3.2 bezeichnet eine Anforderung des Pflichtenhefts B, nicht des Pflichtenhefts A.

2. Testdurchführung

2.1 Testumgebung

Der vollständige Testlauf wurde mit `./mvnw test` (maven-surefire-plugin) ausgeführt.

Merkmal	Wert
Prüfgegenstand	`fakturierung` 1.0.0
Build-/Testwerkzeug	Apache Maven 3.9.9 (Maven-Wrapper), maven-surefire-plugin 3.2.5
Testframework	JUnit Jupiter 5.10.2 (JUnit Platform)
Laufzeitumgebung	Oracle JDK 26 (Build 26+35-2893)
Sprach-/Kompilierziel	Java 21 (`maven.compiler.release=21`)
Betriebssystem	Windows 11 (10.0), amd64
Locale / Kodierung	de_DE; `file.encoding=UTF-8`
Ausführungsdatum	24.06.2026, 08:42 Uhr (Europe/Berlin)
Testkommando	`./mvnw test`

2.2 Vorgehen

Die Testfälle sind deterministisch (feste Ein-/Ausgaben) und mit JUnit 5 umgesetzt. Geldbeträge werden als `java.math.BigDecimal` mit **Scale 2** geprüft. Die jeweils benachbarten Komponenten werden im Modultest durch **Stubs/Mocks** ersetzt:

- **Komponente A** ersetzt die Schnittstellen der Komponenten B (Produkte) und C (Kunden) sowie den PDF-Export durch Stubs.
- **Komponente B** ersetzt die Schnittstelle `ProduktReferenzPruefung` (Komponente A) durch einen Stub.
- **Komponente C** ersetzt die Schnittstelle `KundenReferenzPruefung` (Komponente A) durch einen Stub.
- **Komponente D** ersetzt die Service-Schnittstellen der Komponenten A–C durch Stubs/Mocks.

Die nachfolgenden Tabellen (Abschnitt 3) führen die Testfälle in der Reihenfolge des Modultestplans auf und ergänzen je die ausführende JUnit-Testmethode und das tatsächliche Ergebnis. Bei abweichendem Ergebnis wäre der Befund in Abschnitt 5 dokumentiert.

3. Testergebnisse

3.1 Komponente A — Prozess / Dokumentenzyklus

3.1.1 Dokumentenzyklus (`DokumentzyklusTest`)

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-01	F-23, F-03	`tc01SteuerUndBruttoEinerPosition`	Steuer = 19.00, Brutto = 119.00 (Scale 2)	Bestanden
TC-02	F-23	`tc02Positionssumme`	positionssummeNetto = 150.00	Bestanden
TC-03	F-03, F-13	`tc03BelegSummen`	Netto 200.00, Steuer 32.00, Brutto 232.00	Bestanden
TC-04	F-12, GR-01	`tc04LueckenloseRechnungsnummer`	nächste Nummer `R-2026-000124` (lückenlos)	Bestanden
TC-05	F-12 (Format)	`tc05NummernFormat`	`R-2026-000007` (führende Nullen, `String`)	Bestanden
TC-06	F-14, GR-06	`tc06StandardZahlungsziel`	zahlungsziel = Rechnungsdatum + 14 Tage	Bestanden
TC-07	F-14	`tc07AbweichendesZahlungsziel`	abweichendes Zahlungsziel wird übernommen	Bestanden
TC-08	F-24, NF-INT-01	`tc08UnveraenderlichkeitVersendet`	Änderung wirft `IllegalStateException`	Bestanden
TC-09	F-19, F-20	`tc09Storno`	Status `STORNIERT`; `storniertAm`/`storniertVon` gesetzt	Bestanden
TC-10	F-22, GR-05	`tc10FolgebelegAusAngebot`	AB übernimmt Kunde/Positionen; `vorgaengerNr` gesetzt	Bestanden
TC-11	F-23, F-24	`tc11Snapshot`	einzelpreisNetto bleibt 50.00 (Snapshot unverändert)	Bestanden
TC-12	F-18, NF-USE-02	`tc12PflichtfeldValidierung`	Validierungsfehler benennt fehlendes Pflichtfeld	Bestanden

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-13	F-11, F-12, F-13	`tc13VollstaendigeRechnung`	Rechnung gespeichert; alle § 14 UStG-Pflichtangaben gesetzt	Bestanden

3.1.2 Belegpersistenz (`JsonDokumentRepositoryTest`)

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-14	IF-01 (GoBD)	`belegeWerdenNachNeustartGeladen`	beide Belege nach Neustart geladen; Typen `Rechnung`/`Angebot` erhalten	Bestanden

3.1.3 PDF-Export (`PdfBoxPdfExporterTest`)

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-15	F-15, IF-01	`exportiertVollstaendigenBeleg`	PDF-Datei existiert am Zielpfad	Bestanden
TC-16	F-15 (Robustheit)	`exportiertBelegMitNullPositionsfeldern`	kein Fehler (`assertDoesNotThrow`); PDF existiert	Bestanden
TC-17	F-23 (Robustheit)	`summenberechnungToleriertNullPositionen`	summeNetto = summeSteuer = summeBrutto = 0.00	Bestanden

3.1.4 CSV-Datenexport der Belege (`DokumentCsvExportTest`)

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-18	Q-08, IF-04	`exportiertAlleBelegeMitPositionen`	CSV mit Kopfzeile + je Position eine Zeile (3 Zeilen); enthält `STORNIERT`/Benutzer	Bestanden

3.2 Komponente B — Produktverwaltung

Anforderungs-Nummern beziehen sich auf Teil B des Pflichtenhefts; `ProduktReferenzPruefung` (Komponente A) ist durch einen Stub ersetzt.

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-01	F-01, F-02	`tc01NummernVergabe`	Produkt persistiert; Produktnummer = `P-000042`	Bestanden
TC-02	F-02 (Format)	`tc02NummernFormat`	`P-000007` (führende Nullen, `String`)	Bestanden

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-03	F-03	`tc03NegativerPreis`	Speichern abgelehnt (Validierungsfehler „Einzelpreis“)	Bestanden
TC-04	F-03	`tc04UnzulaessigerSteuersatz`	Steuersatz `0.15` abgelehnt	Bestanden
TC-05	F-04, NF-USE-01	`tc05FehlendeBezeichnung`	abgelehnt; Validierungsfehler benennt „Bezeichnung“	Bestanden
TC-06	F-05	`tc06PreisAendern`	einzelpreisNetto = 95.00 nach Änderung	Bestanden
TC-07	F-07	`tc07ProduktnummerUnveraenderlich`	Änderung der Produktnummer wirft `IllegalArgumentException`	Bestanden
TC-08	F-08	`tc08LoeschenUnverknuepft`	unverknüpftes Produkt entfernt	Bestanden
TC-09	F-09, F-10	`tc09Loeschsperre`	referenziertes Produkt nicht gelöscht (Löschsperre)	Bestanden
TC-10	F-11	`tc10Sortierung`	Reihenfolge „Anker“, „Mast“, „Zaun“	Bestanden
TC-11	F-12	`tc11SucheBezeichnung`	case-insensitive Teilstring-Treffer	Bestanden
TC-12	F-12	`tc12SucheNummer`	Suche nach Produktnummer trifft genau dieses Produkt	Bestanden
TC-13	F-14	`tc13FindeProduktNull`	`findeProdukt("P-999999")` liefert `null`	Bestanden
TC-14	F-15	`tc14CsvExport`	CSV mit Kopfzeile + 3 Datenzeilen, `;`-getrennt, UTF-8	Bestanden

3.3 Komponente C — Kundenverwaltung

Anforderungs-Nummern beziehen sich auf Teil C des Pflichtenhefts; `KundenReferenzPruefung` (Komponente A) ist durch einen Stub ersetzt.

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-01	F-01, F-02	`tc01NummernVergabe`	Kunde persistiert; Kundennummer = `K-000017`	Bestanden
TC-02	F-02 (Format)	`tc02NummernFormat`	`K-000007` (führende Nullen, `String`)	Bestanden
TC-03	F-03, NF-USE-01	`tc03FehlenderOrt`	abgelehnt; Validierungsfehler benennt „Ort“	Bestanden
TC-04	F-03	`tc04LeererName`	leerer Name abgelehnt; benennt „Name“	Bestanden

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-05	F-04	`tc05UngueltigeEMail`	E-Mail `"max.mustermann"` abgelehnt	Bestanden
TC-06	F-04	`tc06GueltigeEMail`	E-Mail `"max@beispiel.de"` gespeichert	Bestanden
TC-07	F-05	`tc07OrtAendern`	Ort „Mannheim“ → „Heidelberg“ gespeichert	Bestanden
TC-08	F-07	`tc08KundennummerUnveraenderlich`	Änderung der Kundennummer wirft `IllegalArgumentException`	Bestanden
TC-09	F-08	`tc09LoeschenUnverknuepft`	unverknüpfter Kunde entfernt	Bestanden
TC-10	F-09, F-10, GR-04	`tc10Loeschsperre`	Löschen abgelehnt; Hinweis enthält Anzahl `3`	Bestanden
TC-11	F-11	`tc11Sortierung`	Reihenfolge „Albrecht“, „Maier“, „Zimmer“	Bestanden
TC-12	F-12	`tc12SucheName`	case-insensitive Teilstring-Treffer	Bestanden
TC-13	F-12, F-14	`tc13SucheNummerUndFindeKunde`	Treffer `K-000017`; `findeKunde` liefert `null` für Unbekannte	Bestanden
TC-14	F-15	`tc14CsvExport`	CSV mit Kopfzeile + 3 Datenzeilen, `;`-getrennt, UTF-8	Bestanden

3.4 Komponente D — Programmoberfläche

Getestet wird die GUI-freie Controller- und Modell-Schicht; die Service-Schnittstellen der Komponenten A–C sind durch Stubs/Mocks ersetzt.

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-01	F-09	`tc01ErsterSchritt`	erster Schritt = `KUNDE_WAEHLEN`	Bestanden
TC-02	F-09	`tc02Schrittfolge`	Schrittfolge bis `SPEICHERN` durchlaufen	Bestanden
TC-03	F-10	`tc03KeinKunde`	Wechsel verhindert; Meldung benennt „Kunde“	Bestanden
TC-04	F-10	`tc04KeinePosition`	Wechsel verhindert; Meldung benennt „Position“	Bestanden
TC-05	F-10	`tc05MengeNull`	Menge 0 verhindert Wechsel; Meldung benennt „Menge“	Bestanden
TC-06	F-11	`tc06ZurueckOhneDatenverlust`	`kundenNr` und `positionen` bleiben erhalten	Bestanden

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
TC-07	F-12	`tc07Zusammenfassung`	enthält Kunde, Positionen, Summen, Datum, Zahlungsziel	Bestanden
TC-08	F-13	`tc08GenauEin Speicheraufruf`	genau ein `erstelle Rechnung(...)`; Erfolgsmeldung mit Nummer	Bestanden
TC-09	F-13 (Fehlerfall)	`tc09Speichern Fehlerfall`	`Meldung(FEHLER, "Rechnungsdatum", …)` dargestellt	Bestanden
TC-10	F-14	`tc10StornierenNur Offen`	*Stornieren* nur bei Status `OFFEN` aktiviert	Bestanden
TC-11	F-15	`tc11StornoNurNach Bestaetigung`	ohne Bestätigung kein Aufruf; mit Bestätigung genau einer	Bestanden
TC-12	F-08	`tc12VersendeterBeleg`	Änderungsaktionen deaktiviert; PDF-Export aktiviert	Bestanden
TC-13	F-06	`tc13Statusfilter`	Statusfilter `OFFEN` liefert genau die 2 offenen Belege	Bestanden
TC-14	F-03	`tc14StammdatenSuche`	Controller delegiert an `KundenService.suche(...)`; Treffer	Bestanden
TC-15	F-03 (D-F-03)	`tc15KundenListe`	leerer Suchbegriff: gesamter Bestand; sonst gefiltert	Bestanden

3.5 Gemeinsame Infrastruktur (Paket `gemeinsam`)

Querschnittliche Dienste, die von allen Komponenten genutzt werden: der `EreignisBus` (Observer-Muster) sowie die atomare `JsonPersistenz` hinter allen Repositories (IF-01).

3.5.1 Ereignisbenachrichtigung (`EreignisBusTest`)

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
INF-01	Observer (PH-A §7)	`benachrichtigtAlle Beobachter`	beide Beobachter genau 2× benachrichtigt	Bestanden
INF-02	Observer	`benachrichtigtNur BetroffenenBereich`	nur `DOKUMENTE`-Beobachter; `KUNDEN` 0×	Bestanden
INF-03	Observer (Robustheit)	`meldenOhneBeobachter IstWirkungslos`	wirkungslos, keine Exception	Bestanden

3.5.2 Atomare JSON-Persistenz (`JsonPersistenzTest`)

TC	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis	Status
INF-04	IF-01 (atomar)	`schreibtOhneTempDateiRueckstand`	Zieldatei existiert; keine `.tmp`-Restdatei; Inhalt korrekt	Bestanden
INF-05	IF-01	`ersetztVorhandenenBestand`	Bestand vollständig ersetzt	Bestanden
INF-06	IF-01	`legtElternverzeichnisseAn`	fehlende Elternverzeichnisse angelegt; Datei existiert	Bestanden

3.6 Performance-/Lastnachweise (`PerformanceTest`)

Übergreifende Lastnachweise gemäß Lastenheft (Referenzgröße: 5.000 Kunden, 5.000 Produkte, 1.000 Belege; das Befüllen fließt nicht in die Messung ein). Die Spalte *Status* nennt neben dem Ergebnis die gemessene Ausführungszeit (Surefire) gegen die jeweilige Zeitschranke.

ID	Abgedeckte Anf.	JUnit-Testmethode	Erwartetes Ergebnis (Zeitschranke)	Status
Q-04	Q-04 (Start)	`q04Anwendungsstart`	drei Repositories laden in ≤ 5 s	Bestanden (gemessen 0,039 s)
Q-02	Q-02 (Suche)	`q02Suche`	Suche/Auflistung in ≤ 1 s	Bestanden (gemessen 0,025 s)
Q-03	Q-03 (PDF)	`q03PdfErstellung`	PDF-Erstellung (50 Positionen) in ≤ 2 s	Bestanden (gemessen 0,011 s)
Q-08	Q-08 (Export)	`q08Datenexport`	Vollexport (CSV) in ≤ 30 s	Bestanden (gemessen 0,181 s)

4. Ergebnisübersicht und Statistik

Testklasse	Komponente	Anzahl	Bestanden	Fehlgeschlagen
`dokumente/DokumentzyklusTest`	A – Prozess/Dokumentenzyklus	13	13	0
`dokumente/JsonDokumentRepositoryTest`	A – Belegpersistenz (IF-01/GoBD)	1	1	0
`dokumente/PdfBoxPdfExporterTest`	A – PDF-Export (F-15)	3	3	0

Testklasse	Komponente	Anzahl	Bestanden	Fehlgeschlagen
`dokumente/DokumentCsvExportTest`	A – CSV-Export Belege (Q-08)	1	1	0
`produkte/ProduktVerwaltungTest`	B – Produktverwaltung	14	14	0
`kunden/KundenVerwaltungTest`	C – Kundenverwaltung	14	14	0
`gui/OberflaechenControllerTest`	D – Programmoberfläche	15	15	0
`gemeinsam/EreignisBusTest`	Gemeinsame Infrastruktur (Observer)	3	3	0
`gemeinsam/JsonPersistenzTest`	Gemeinsame Infrastruktur (Persistenz)	3	3	0
`PerformanceTest`	Querschnitt (Q-02/Q-03/Q-04/Q-08)	4	4	0
Summe		**71**	**71**	**0**

Surefire-Gesamtergebnis: `Tests run: 71, Failures: 0, Errors: 0, Skipped: 0`. Reine Testausführung (Surefire): ≈ 1,4 s; Maven-Gesamtlauf: ≈ 3,1 s. Die **Bestehensquote beträgt 100 %** (71 von 71 Testfällen bestanden).

5. Nachtest Version 3.0 (24.09.2026)

Zum Abschluss der Version 3.0 wurde die gesamte Testsuite erneut ausgeführt — diesmal über `./mvnw verify`, also einschließlich der Abdeckungsschwelle (JaCoCo) und der statischen Analyse (SpotBugs), die seit Version 2.0 bzw. 3.0 Teil des Builds sind. Die Testfälle der Version 1.0 (Kapitel 3) sind darin vollständig enthalten; die ergänzten Testfälle sind im Modultestplan, Kapitel 3, spezifiziert.

5.1 Testumgebung

Merkmal	Wert
Prüfgegenstand	`faktura` 3.0.0
Build-/Testwerkzeug	Apache Maven 3.9.9 (Maven-Wrapper), maven-surefire-plugin 3.2.5
Laufzeit	JDK 26.0.1 lokal (übersetzt mit `--release 21`); CI: Temurin 21 unter Ubuntu und Windows
Testframework	JUnit Jupiter (über die Spring-Boot-Stückliste)

Merkmal	Wert
Qualitätsschranken	JaCoCo 0.8.15 (≥ 85 % Anweisungen, ≥ 70 % Zweige), SpotBugs 4.10.3

5.2 Ergebnisse je Testklasse

Testklasse	Bereich	Ausführungen	Fehlgeschlagen	Dauer (s)
`dokumente/DokumentCsvExportTest`	A – Dokumentenzyklus	1	0	0.53
`dokumente/DokumentzyklusTest`	A – Dokumentenzyklus	23	0	0.28
`dokumente/ERechnungExportTest`	A – Dokumentenzyklus	8	0	0.42
`dokumente/JdbcDokumentRepositoryTest`	A – Dokumentenzyklus	7	0	1.34
`dokumente/JsonDokumentRepositoryTest`	A – Dokumentenzyklus	1	0	0.07
`dokumente/KennzahlenDienstTest`	A – Dokumentenzyklus	6	0	0.01
`dokumente/NummernkreisIntegritaetTest`	A – Dokumentenzyklus	3	0	2.61
`dokumente/PdfBoxPdfExporterTest`	A – Dokumentenzyklus	9	0	0.50
`produkte/ProduktVerwaltungTest`	B – Produktverwaltung	16	0	0.08
`kunden/KundenVerwaltungTest`	C – Kundenverwaltung	20	0	0.11
`gui/DokumentListenControllerTest`	D – Programmoberfläche	6	0	0.01
`gui/OberflaechenControllerTest`	D – Programmoberfläche	17	0	0.03
`gui/TabellenFormatTest`	D – Programmoberfläche	16	0	0.03

Testklasse	Bereich	Ausführungen	Fehlgeschlagen	Dauer (s)
`gemeinsam/CsvTest`	Gemeinsame Infrastruktur	3	0	0.01
`gemeinsam/DatensicherungTest`	Gemeinsame Infrastruktur	2	0	0.40
`gemeinsam/EreignisBusTest`	Gemeinsame Infrastruktur	3	0	0.00
`gemeinsam/JsonPersistenzTest`	Gemeinsame Infrastruktur	3	0	0.04
`gemeinsam/ValidierungTest`	Gemeinsame Infrastruktur	38	0	0.11
`firma/FirmenprofilServiceTest`	Firmenprofil	9	0	1.37
`FakturaApplicationTest`	Anwendung/Querschnitt	6	0	1.52
`PerformanceTest`	Anwendung/Querschnitt	4	0	0.62
Summe		**201**	**0**	

5.3 Qualitätsschranken

Prüfung	Schranke	Ergebnis
Anweisungsabdeckung (nicht-grafischer Code)	≥ 85 %	87 % — eingehalten
Zweigabdeckung (nicht-grafischer Code)	≥ 70 %	73 % — eingehalten
SpotBugs	0 Befunde	0 Befunde

5.4 Ergänzende Prüfung der Oberfläche

Die Oberfläche selbst ist nicht Gegenstand automatisierter Tests (Kapitel 6). Zum Abschluss der Version 3.0 wurde sie mit einem fiktiven Demodatenbestand (8 Kunden, 8 Produkte, 17 Belege) manuell durchlaufen: Übersicht und Kennzahlen, Statusfilter, Zahlungseingang, Storno einer versendeten Rechnung mit Stornorechnung, Firmenprofil, Tastenkürzel nach Ansichtswechsel und der vollständige Rechnungswizard mit 7 %/19 %-Positionen. Die dabei gefundenen Darstellungsfehler (abgeschnittene Detailangaben, fehlender Bezahlt-Status in der Übersicht, Sortierung von Namen mit „Ł“) wurden behoben; die erzeugten PDF- und E-Rechnungsdateien wurden gegeneinander geprüft (identische Summen).

6. Bewertung und Abweichungen

Abweichungen: keine. Alle **71** im Modultestplan v2.0 spezifizierten Testfälle wurden ausgeführt und sind bestanden; es traten weder Fehlschläge (*Failures*) noch Fehler (*Errors*) auf, kein Testfall wurde übersprungen.

Performance: Alle vier Lastnachweise (Q-02, Q-03, Q-04, Q-08) wurden deutlich innerhalb der im Lastenheft geforderten Zeitschranken erfüllt (gemessene Ausführungszeiten ≤ 0,2 s gegenüber Schranken von 1 s bis 30 s).

Hinweis zum Prüfumfang: Die Programmoberfläche (Komponente D) wurde — wie im Modultestplan vorgesehen — auf Ebene der Controller- und Modell-Schicht ohne Swing-Rendering geprüft; die Service-Schnittstellen der Nachbarkomponenten wurden durch Stubs/Mocks ersetzt.

Gesamturteil: Der Modultest des Gesamtsystems *Desktop-Fakturierungsanwendung* gilt als **bestanden**. Die im Modultestplan festgelegten funktionalen Kernregeln, die zentralen Geschäftsregeln (GR-01…GR-06) sowie die Qualitäts-/Performanceanforderungen (Q-02, Q-03, Q-04, Q-08, Q-09) sind nachgewiesen.

7. Abkürzungen

Abkürzung	Bedeutung
TC	Testfall (Test Case)
INF	Infrastruktur-Testfall (Paket `gemeinsam`)
F	Funktionale Anforderung (Pflichtenheft, komponentenlokal)
NF	Nicht-funktionale Anforderung (Pflichtenheft)
IF	Schnittstelle (Interface)
GR	Geschäftsregel (Lastenheft)
Q	Qualitätsanforderung (Lastenheft)
PH	Pflichtenheft
CSV	Comma-Separated Values (offenes Exportformat)
GoBD	Grundsätze zur ordnungsmäßigen Führung und Aufbewahrung von Büchern
SUT	System under Test (Prüfgegenstand)
Surefire	Maven-Plugin zur Testausführung (Berichte unter `target/surefire-reports/`)
Bestanden	Testfall erfolgreich: tatsächliches = erwartetes Ergebnis
Fehlgeschlagen	Testfall nicht erfolgreich (*Failure* oder *Error*)

8. Review-Remediation 3.1.0-rc.1 (10.10.2026)

8.1 Gegenstand und Umgebung

Dieser Lauf prüft Preiseingabe, kalenderstrenge Datumsfelder, persistente Betrags- und Einheitensnapshots, V4-Upgrade und ältere JSON-Importe, PDF/XML/CSV, Einstellungen, Sicherung und Wiederherstellung. V1 bis V4 der Migrationen bleiben unverändert. Die historischen Kapitel 1–6 werden nicht als Nachweis der neuen Funktionen gewertet.

Lokale Umgebung: Windows 11, Build 10.0.26300, x64; AMD Ryzen 5 7530U, 12 logische Prozessoren, rund 16 GB RAM; Temurin 21.0.12.1+1; Maven 3.9.9. Die Maschine ist ein Entwicklungsrechner, kein disponibler CI-Runner.

8.2 Funktionale und technische Regressionen

Prüfbefehl: `mvnw.cmd verify -Dfaktura.gui.tests=true` mit JDK 21. Ergebnis (Stand 11.10.2026, nach den Review-Korrekturen): **224 Testfälle, 223 ausgeführt, 0 Fehler, 1 bewusst separat aktivierbarer Leistungstest übersprungen; BUILD SUCCESS**. JaCoCo-Schwellen erfüllt, SpotBugs ohne Befund. Die Ergebnisdatei ist `target/verify-abschluss.log`; die einzelnen Testfälle stehen in `target/surefire-reports`. Die Produktions-Leistungsmessung wird getrennt und ohne JaCoCo aktiviert. Die Abdeckungsschwellen bleiben 85 % Anweisungen und 70 % Zweige.

Zusätzlich besteht ERE-PAKET-01 unter Windows: Dasselbe gepackte JAR erzeugt gültiges EN-16931-CII, führt Schema- und Schematronregeln aus und lehnt einen fehlenden Käufernamen ab, ohne das bestehende XML-Ziel zu überschreiben. Die Ressourcenauflösung funktioniert auch aus verschachtelten JARs. ERE-12 sperrt fremde Schemaquellen und Pfadmanipulation. Unbekannte Positionssteuer und Positionsbrutto bleiben auch im CSV leer; TC-17 erwartet ab 3.1 unbekannte Summen statt der historischen Nullersetzung. GUI-REM-03 deckt sowohl fachliche Validierungsfehler als auch einen echten Datenbankfehler beim Speichern ab; das Schließen bleibt in beiden Fällen gesperrt.

REM-03 hält den historischen Betrag 0,08 EUR über Speicherung, PDF, Stornierung zu -0,08 EUR und Neustart unverändert. REM-04/07/08 prüfen V4, ältere JSON-Daten, fehlende Felder sowie eine fehlgeschlagene Upgradeprobe mit erfolgreichem Wiederholungsversuch. REM-09 prüft die Einheitenkopie durch den gesamten Belegzyklus nach einer Produktänderung. REM-10 zeigt, dass Produkte ohne Einheit Positionen mit Stück/C62 erzeugen und die neuen Codes LS (Pauschale) und MON (Monat) die Mustang-Prüfung bestehen. REM-11 belegt, dass die eingefrorene V6-Migration Altbestände genauso einordnet wie `Betragssnapshot` und die Steuerzeilen schreibt. REM-12: Nicht entfernbare Temp-Verzeichnisse erzeugen nur eine Warnung. GUI-REM-04 prüft die Rückfrage beim Beenden, während die Einstellungen nicht angezeigt werden (zuvor NullPointerException im Schließen-Ereignis).

PDF-REM-01 erzeugt normale, millionenschwere und extrem große Beträge mit Langtexten und freien Einheiten. Alle gerenderten Seiten werden visuell kontrolliert; Zeichenkoordinaten prüfen zusätzlich seitliche Grenzen und Überlappungen. Prüfbilder liegen unter `target/pdf-pruefung`. Die separat gemessenen Summen bleiben vollständig lesbar; bei ungeklärter Historie steht die fehlende Steueraufschlüsselung ausdrücklich im PDF. PDF-REM-02 druckt Altpositionen ohne Einheit und ohne Produktnummer ohne zusätzlichen Hinweistext und prüft für 40 mehrzeilige Positionen in Tabellen- und Stapellayout, dass keine Position über einen Seitenwechsel getrennt wird.

ERE-PAKET-01 prüft im gepackten JAR zusätzlich, dass Flyway alle Migrationen V1–V6 einschließlich der Java-Migration findet; ein Smoke-Start desselben JARs wendete auf einer leeren Datenbank sechs Migrationen an und erreichte den Readiness-Marker nach 4,0 s.

8.3 Produktionspfade und Startzeit

SQLite-Referenzbestand: 5.000 Kunden, 5.000 Produkte und 1.000 Belege mit je fünf Positionen. Die abschließende Messung vom 10.10.2026 nutzt Spring-Dienste und die produktiven Exporte, ohne JaCoCo. Aufbau des Bestands liegt außerhalb der Messung.

Operation	Gemessen	Grenze	Lokales Ergebnis
Kundensuche	69 ms	1.000 ms	bestanden
Produktsuche	34 ms	1.000 ms	bestanden
Belegliste	104 ms	1.000 ms	bestanden
Dashboard	80 ms	1.000 ms	bestanden
PDF mit 50 Positionen	379 ms	2.000 ms	bestanden
Gesamt-ZIP	575 ms	30.000 ms	bestanden

Erster gepackter Startlauf vom 10.10.2026: **11.730 / 10.462 / 9.337 ms**. Alle drei Läufe überschreiten 5.000 ms. Eine geerbte DEBUG-Umgebungsvariable aktivierte zusätzliche Spring-Ausgaben; das Messskript entfernt sie jetzt aus dem Kindprozess. Die bereinigte Serie ergab **6.711 / 5.271 / 9.490 ms**; auch sie erfüllt die Freigabegrenze nicht. Die ursprünglichen Messungen bleiben erhalten. Das Skript archiviert neue Serien in eigenen Zeitstempelordnern und misst weiterhin vom Prozessstart bis nach Fensteranzeige und erstem JavaFX-Ereigniszyklus. Ein Profilinglauf bestätigt Aufwand in JVM-/Klassenladen, Spring-Initialisierung und JavaFX-Layout; er wird nicht als Freigabemessung gewertet.

Nach Beseitigung der doppelten Flyway-Metadatenabfrage ergab die letzte Serie **4.709 / 5.085 / 5.972 ms**. Ein Lauf unterschreitet die Grenze; zwei überschreiten sie. Die RC-Freigabe bleibt deshalb offen. Alle Serien bleiben im Prüfverzeichnis erhalten.

Serien vom 11.10.2026 (gleicher Code, Referenzbestand, je drei frische Starts):

Umgebung	Laufzeit	Starts (ms)
Entwicklungsrechner (siehe 8.1)	System-Java 26.0.1	4.502 / 4.964 / 5.270
GitHub-Runner, AMD EPYC 9V45, 4 Kerne	Temurin 21	4.642 / 3.929 / 4.522
GitHub-Runner, AMD EPYC 7763, 4 Kerne	Temurin 21	6.609 / 6.250 / 5.782

Q-04 ist damit **nicht zuverlässig erfüllt**. Die Zeit verteilt sich auf JVM-Start und Entpacken des Fat-JARs (≈ 1,2–1,6 s), den Spring-Kontext (1,8–2,8 s, mit dem Bestand wachsend) und den JavaFX-Aufbau bis zum Readiness-Marker (≈ 1,4–2,2 s). Die Grenze von 5 Sekunden bleibt unverändert. Weil gehostete Runner zufällig unterschiedliche CPUs erhalten, prüft der Release-Workflow die Startzeit nur noch als Regressionswächter mit 8 Sekunden. Version 3.1.0-rc.1 erscheint als Vorabversion mit dieser dokumentierten Lücke; die endgültige Version 3.1.0 bleibt gesperrt, bis Q-04 mit der gebündelten Laufzeit auf dem Referenzrechner eingehalten wird.

8.4 Offene Freigabeschranken

- Die Startzeitgrenze ist lokal nicht nachgewiesen. Der dedizierte RC-Job muss drei frische Starts und alle Produktionsoperationen mit den unveränderten Grenzwerten bestehen. *Nachtrag 18.10.2026:* mit der gebündelten Laufzeit nachgewiesen, siehe Kapitel 9.

- Windows-JAR-Start und gepackte XML-Prüfung wurden lokal ausgeführt. Linux x64 und macOS arm64 mit demselben JAR sowie JDK 21 sind automatisiert eingerichtet, aber hier nicht ausgeführt. Die Plattformmatrix ist noch kein erfolgreicher Plattformnachweis.
- Der MSI-Test installiert, startet mit gebündelter Laufzeit, prüft isolierte Daten und deinstalliert ausschließlich auf einem disponiblen Windows-CI-Runner. Er wurde auf dem Entwicklungsrechner bewusst nicht als Ersatz für diesen Nachweis ausgeführt.
- Der volle Dependency-Check 12.2.2 wurde am 10.10.2026 gestartet und scheiterte beim NVD-Update mit HTTP 429. Damit liegt **kein vollständiger Sicherheitsnachweis** vor. CI muss den Scan mit NVD-API-Schlüssel erfolgreich ausführen; High/Critical blockieren. Es gibt keine dokumentierte Ausnahme für offene Befunde.
- Usability-Studien mit mindestens fünf Personen sind auf Wunsch des Projektverantwortlichen derzeit zurückgestellt; die Kriterien bleiben unbewiesen. Eine rechtliche Gesamtbewertung ist nicht durchgeführt und gehört nicht zum Portfolio-Abschluss.
- Der in Q-06 spezifizierte Netzwerk-Monitoring-Nachweis ist noch nicht dokumentiert.

Es wurde kein Release veröffentlicht oder getaggt. Produktionsdatenbanken wurden nicht verändert; Migrationen, Sicherungen und Fehlerfälle liefen ausschließlich mit Testbeständen.

8.5 Portfolio-Abschlussrunde (10.10.2026)

PORT-01 wurde mit `tools/portfolio-demo.ps1` gegen das gebaute 3.1.0-rc.1-JAR erfolgreich ausgeführt. Es entstanden ausschließlich fiktive Daten: ein Kunde, zwei Produkte, fünf Belege, Beispiel-PDF, schema-/schematrongeprüftes XML, Gesamt-CSV-ZIP und Sicherung. Eine Produktpreisänderung von 80,00 auf 95,00 EUR änderte die bestehende Rechnung nicht: 185,00 EUR netto, 35,15 EUR Steuer und 220,15 EUR brutto bleiben erhalten. Anschließend wurde die JavaFX-Anwendung mit einer Kopie dieses Bestands gestartet; Bereitschaftsmarker, erfolgreicher Prozessabschluss und isolierte Datenbank wurden geprüft. Das ist eine technische Demo-Prüfung, keine Usability-Studie oder aufgezeichnete Vorführung. Die beiden Demo-ZIPs bestanden außerdem die CRC-Prüfung. Eine aus der Sicherung in eine neue Datei wiederhergestellte Datenbank bestand `integrity_check` und `foreign_key_check`; alle sieben Geschäftstabellen einschließlich Nummernkreisen stimmten mit dem Ausgangsbestand überein. Protokoll: `target/portfolio-ausgaben-pruefung.log`.

PORT-02 bestand lokal auf Windows 11 x64: `jpackage --type app-image` erzeugte ein portables Paket mit Temurin 21.0.12.1+1. Das offizielle JDK-Archiv wurde vor dem Entpacken gegen die veröffentlichte SHA-256 geprüft und ausschließlich im Build-Verzeichnis abgelegt. Die EXE startete mit ihrer gebündelten Laufzeit, ohne JAVA_HOME und mit isoliertem Datenpfad. Die Testdatenbank und der Bereitschaftsmarker waren vorhanden; im Programmverzeichnis lag keine Datenbank. Dies belegt weder MSI-Installation noch die 5-Sekunden-Startgrenze.

Geprüftes JAR (SHA-256): `9AD2AEDE4D421A126731720C2E1F974F1E7AB201332DA01957A9A01F4E92CC92`. Lokale Protokolle: `target/portfolio-demo.log`, `target/portfolio-start.log`, `target/windows-paket.log`; das Paketverzeichnis enthält zusätzlich `pruefbericht.txt`. Die damals 219 JUnit-Fälle aus Kapitel 8.2 wurden durch diese zwei separaten Prüfprogramme nicht erweitert. Anwendungscode wurde in dieser Portfolio-Runde nicht geändert.

Der erneute vollständige Dependency-Check scheiterte nach 4:25 Minuten erneut mit NVD HTTP 429 und fehlender vollständiger Datenbasis (`target/dependency-scan-portfolio.log`). Es gibt weiterhin keinen erfolgreichen Vollscan. GitHub-Anmeldung ist vorhanden, aber kein NVD_API_KEY-Secret und kein Remote-Stand dieser neuen Workflows. Der unabhängige manuelle Workflow `portfolio.yml` ist lokal vorbereitet; er veröffentlicht nichts. MSI- und externe Plattformnachweise bleiben offen. Es wurde nichts gepusht, installiert, getaggt oder veröffentlicht.

Die Portfolio-Abgrenzung benennt zurückgestellte Usability, spätere Produktfunktionen sowie die verbleibenden Abschlussnachweise.

9. Startzeit Q-04 vor der Freigabe 3.1.0 (18.10.2026)

9.1 Gegenstand und Umgebung

Gemessen wird Q-04 (NF-PERF-01): Hauptfenster bedienbereit in höchstens 5 Sekunden mit dem Referenzbestand aus 8.3 (5.000 Kunden, 5.000 Produkte, 1.000 Belege mit je fünf Positionen). Rechner wie in 8.1, am Netzteil. Im Akkubetrieb dauerten dieselben Starts etwa doppelt so lange; solche Messungen sind nicht vergleichbar und wurden verworfen.

Laufzeit ist die **gebündelte Java-Laufzeit des veröffentlichten 3.1.0-rc.1-MSI** (Temurin 21.0.12.1), administrativ entpackt (`msiexec /a`), also ohne Installation auf dem Entwicklungsrechner. `tools/paket-pruefen.ps1` misst wie bisher vom Prozessstart bis zum Bereitschaftsmarker nach Fensteranzeige und erstem JavaFX-Ereigniszyklus und startet jetzt wahlweise den Starter `Faktura.exe` (`-Starter`) statt `java -jar`. Jeder Start schreibt zusätzlich die Zeile `FAKTURA_STARTPHASEN` mit den Phasen in Millisekunden seit Prozessstart (`Startphasen`); das Skript übernimmt sie in `startzeiten.csv`. Änderungen wurden in abwechselnden A/B-Serien verglichen, weil derselbe Stand zu verschiedenen Zeiten um bis zu 0,4 s streut.

9.2 Ausgangslage

3.1.0-rc.1 wie veröffentlicht: **5.834 / 4.536 / 4.923 ms**. Aufteilung eines Starts: rund 0,2 s bis `main`, 0,5 s JavaFX-Laufzeit, 2,3 s Spring-Kontext, 0,6 s Szenenaufbau einschließlich Übersicht und 0,6 s Fensteranzeige. Eine Auswertung des Klassenladens (`-Xlog:class+load`) zeigt rund 10.300 geladene Klassen ohne nennenswerte Wartezeiten: Der Start ist durchgehend durch Klassenladen bestimmt. Allein Flyway und das von Flyway für seine Konfiguration genutzte Jackson laden rund 950 Klassen, obwohl beim Start eines aktuellen Bestands nichts zu migrieren ist.

9.3 Maßnahmen

Maßnahme	Vorher	Nachher	Wirkung
Paket enthält das entpackte Spring-Boot-Layout statt des Fat-JARs (`tools/windows-paket-bauen.ps1`)	4.445–4.688 ms	3.640–3.739 ms	≈ −0,9 s
Flyway entfällt, wenn der Bestand auf der ausgelieferten Version steht (`Schemastand`)	Median 3.995 ms	Median 3.561 ms	≈ −0,4 s

Maßnahme	Vorher	Nachher	Wirkung
Übersicht wertet den Belegbestand im Hintergrund aus (`HintergrundAufgaben`)	Median 3.502 ms	Median 3.356 ms	≈ −0,15 s

Das veröffentlichte Fat-JAR bleibt unverändert; das MSI enthält dasselbe geprüfte JAR, entpackt in Anwendungs-JAR und `lib/`. Jede Abweichung vom ausgelieferten Schemastand (neuer Bestand, ausstehende, fehlgeschlagene oder neuere Migration) läuft weiterhin über Flyway samt Upgradeprobe (Q-06). Die Übersicht zeigt bis zum Ergebnis Platzhalterkarten; das Fenster ist in dieser Zeit bedienbar.

9.4 Geprüfte und verworfene Ansätze

Ansatz	Ergebnis	Entscheidung
Anwendungs-CDS-Archiv aus einem Trainingslauf	2,3 s je Start, aber an den absoluten Installationspfad gebunden; nach dem Verschieben meldet Java 21 „Required classpath entry does not exist“ und startet ohne Archiv	verworfen
`-XX:+AutoCreateSharedArchive` (Archiv entsteht beim Anwender)	Aufzeichnungslauf nach jeder Installation rund 20 s; ein mitgelieferter Platzhalter wird nicht ersetzt	verworfen
CDS-Archiv der JDK-Klassen (`jlink --generate-cds-archive`)	Median 3.357 gegenüber 3.403 ms, also im Rauschen; rund 30 MB mehr und unverträglich mit `--strip-native-commands`	verworfen
Spring-Kontext parallel zur JavaFX-Laufzeit starten	3.600–4.218 gegenüber 3.640–3.739 ms; beide konkurrieren ums Klassenladen	verworfen
Logging-System von Spring Boot abschalten	Median 3.989 gegenüber 3.959 ms	verworfen

Ein Java-25-AOT-Cache wurde nicht untersucht; die gebündelte Laufzeit bleibt Java 21.

9.5 Nachweis

Paket wie in der Release-Pipeline: gebündelte Laufzeit 21.0.12.1 und der von `tools/windows-paket-bauen.ps1` erzeugte Anwendungsordner. Drei frische Starts mit der unveränderten Grenze von 5.000 ms:

Start	Gesamt	bis `main`	JavaFX bereit	Spring-Kontext	Szene	Fenster	Bereit
1	3.730 ms	122	841	2.673	3.155	3.656	3.707
2	3.699 ms	136	818	2.644	3.113	3.620	3.674

Start	Gesamt	bis `main`	JavaFX bereit	Spring-Kontext	Szene	Fenster	Bereit
3	3.536 ms	135	827	2.547	2.960	3.451	3.507

Q-04 ist damit **bestanden**. Eine anschließende Serie aus zehn Starts lag zwischen 3.328 und 4.166 ms (Median 3.663 ms). Der allererste Start nach dem Kopieren frischer Dateien dauerte 7,0 s (Virenscan, leerer Dateicache); dasselbe ist für den ersten Start nach einer Installation zu erwarten. Q-04 misst wie bisher wiederholte Starts mit jeweils frischem Datenverzeichnis. Protokolle: `target/paketpruefung/20261018-203635-409` und `20261018-203651-134`. Der Release-Workflow behält die Startzeit als Regressionswächter mit 8 Sekunden (gehostete Runner, siehe 8.3).

9.6 Testfälle und Qualitätsschranken

Neu: SP-01 und SP-02 (`StartphasenTest`), SST-01 bis SST-05 (`SchemastandTest`). `./mvnw verify`: 232 Ausführungen, 0 Fehlschläge, 5 übersprungen (GUI- und Produktionsleistungstests nur auf Schalter), Abdeckungsschwellen und SpotBugs ohne Befund. `GuiRegressionTest` mit `-Dfaktura.gui.tests=true`: 4 von 4 bestanden. Die Übersicht wurde mit dem Referenzbestand im Ladezustand und gefüllt geprüft, ebenso mit einer Rechnung ohne gespeicherte Summen: Offener Betrag und Umsatz zeigen „unbekannt“ samt Anzahl.

9.7 Offene Punkte

- Das MSI mit dem neuen Layout wird ausschließlich auf einem disponiblen Windows-Runner installiert, gestartet und entfernt (`tools/msi-pruefen.ps1`); dieser Lauf steht aus.
- Upgrade von 3.1.0-rc.1 auf 3.1.0 mit dem Installer (beide MSI-Version 3.1.0) ist noch nicht geprüft.
- In über 100 lokalen Starts endete einer nach dem Bereitschaftsmarker mit einem Exitcode ungleich 0; zehn gezielte Wiederholungen blieben fehlerfrei. Das Messskript meldet den Exitcode jetzt ausdrücklich.